Experiment 05: Containerizing Applications using Docker

Experiment No.	05
Mapped CO	CO4
Aim	To containerize an application by writing a Dockerfile, building an image and running it as a container.
Tools / Software	Docker Engine; A sample web application such as Python Flask or Node.js; Docker Hub account optional; Code editor / terminal

Theory

A container is an isolated and runnable instance of an application packaged with its required dependencies. Containers share the host operating-system kernel and therefore start faster and consume fewer resources than traditional virtual machines. Docker is a commonly used platform for building, running and managing containers.

A Docker image is an immutable, layered template created from instructions written in a Dockerfile. Common Dockerfile instructions include FROM, WORKDIR, COPY, RUN, EXPOSE and CMD. When an image is executed, it becomes a container. Images can be tagged and stored in a registry such as Docker Hub for sharing and deployment.

Procedure

1. Install Docker Engine and verify the installation using docker --version.
2. Create a project directory containing a sample web application file and a dependency file such as requirements.txt.
3. Write a Dockerfile defining the base image, working directory, dependency installation, port exposure and start command.
4. Build the Docker image using docker build and assign a meaningful tag such as myapp:1.0.
5. Run the image as a container using docker run and map the container port to a host port.
6. List running containers using docker ps and inspect container logs using docker logs.
7. Open the application in a browser or use curl to verify that it is reachable on the mapped port.
8. Optionally tag and push the image to Docker Hub for sharing.

Sample Code / Commands

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello from Docker Container"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
flask
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build, run and publish commands

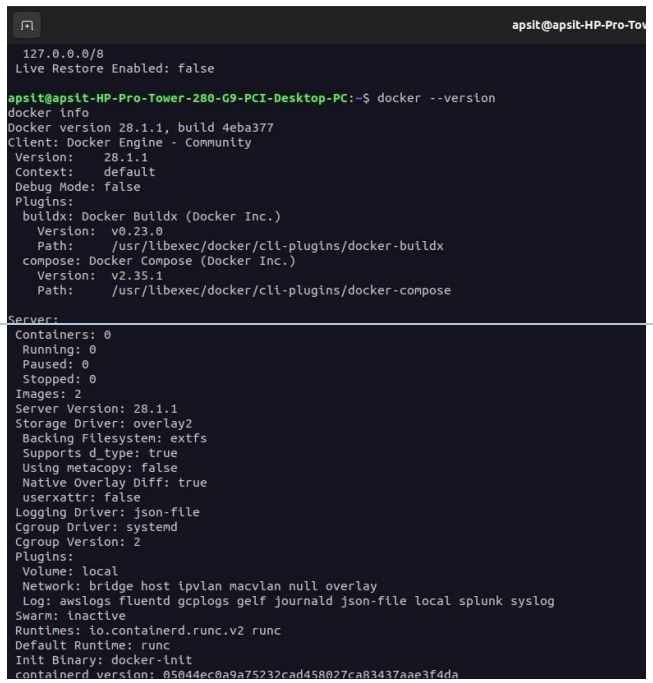
```
docker --version
docker build -t myapp:1.0 .
docker run -d -p 5000:5000 --name myapp myapp:1.0
docker ps
docker logs myapp
curl http://localhost:5000
docker tag myapp:1.0 <user>/myapp:1.0
docker push <user>/myapp:1.0
```

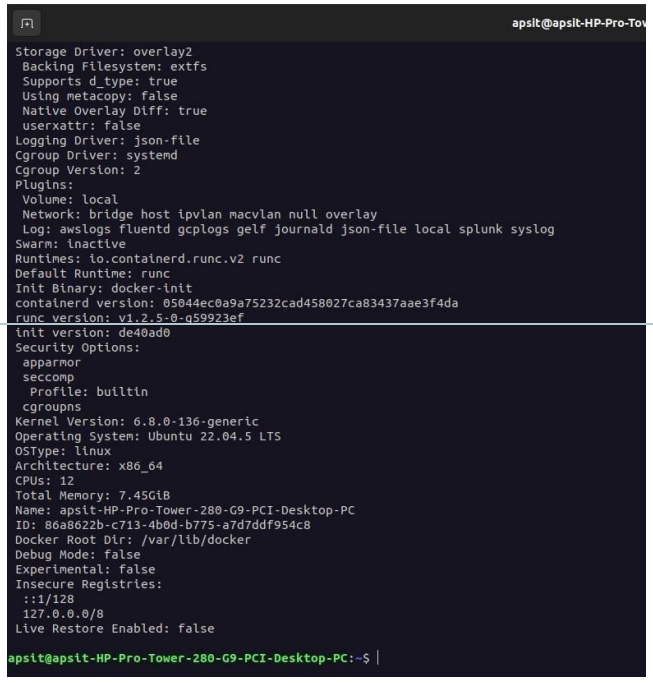
Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing Docker installation verification, application creation, image building, container execution and output validation. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Gargee Warke	65	B2	14/08/26

1. Docker Installation and Version Output





2. Application Source and Dependency Files Output

```
 CPUs: 12
 Total Memory: 7.45GiB
 Name: apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC
 ID: 86a8622b-c713-4b0d-b775-a7d7ddf954c8
 Docker Root Dir: /var/lib/docker
 Debug Mode: false
 Experimental: false
 Insecure Registries:
  ::1/128
  127.0.0.0/8
 Live Restore Enabled: false

apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mkdir -p ~/docker-flask-demo
cd ~/docker-flask-demo
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ pwd
/home/apsit/docker-flask-demo
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ ls -la
total 16
drwxrwxr-x  2 apsit docker  4096 Aug 14 09:16 .
drwxr-x--- 67 apsit apsit  12288 Aug 14 09:16 ..
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ nano app.py
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ nano requirements.t
xt
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ ls -l
total 8
-rw-rw-r-- 1 apsit docker 184 Aug 14 09:22 app.py
-rw-rw-r-- 1 apsit docker   6 Aug 14 09:23 requirements.txt
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ cat app.py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Hello from Docker Container"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ cat requirements.tx
t
flask
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$
```

3. Dockerfile Output

```
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ nano Dockerfile
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ cat Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ ls -l
total 12
-rw-rw-r-- 1 apsit docker 184 Aug 14 09:22 app.py
-rw-rw-r-- 1 apsit docker 156 Aug 14 09:25 Dockerfile
-rw-rw-r-- 1 apsit docker   6 Aug 14 09:23 requirements.txt
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$
```

4. Docker Image Build Output

```
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ docker build -t mya
pp:1.0 .
[+] Building 42.5s (10/10) FINISHED                                 docker:default
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$
```

```
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED          SIZE
myapp         1.0       331124178143   2 minutes ago    132MB
ubuntu        latest    f794f40ddfff   5 months ago     78.1MB
hello-world   latest    1b44b5a3e06a   12 months ago    10.1kB
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$
```

5. Container Run, Logs and Application Access Output

```
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ docker ps
docker logs myapp
curl http://localhost:5000
CONTAINER ID   IMAGE       COMMAND           CREATED         STATUS         PORTS
                                  NAMES
ca069fa8809e   myapp:1.0   "python app.py"   2 minutes ago   Up 2 minutes   0.0.0.0:500
0->5000/tcp, [::]:5000->5000/tcp   myapp
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a
production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [14/Aug/2026 04:04:16] "GET / HTTP/1.1" 200 -
Hello from Docker Containerapsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flas
k-demo$
```

6. Docker Hub Push Output (Optional)

```
Hello from Docker Containerapsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flas
k-demo$ docker stop myapp
myapp
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ docker rm myapp
myapp
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
apsit@apsit-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/docker-flask-demo$
```

Final Observation Statement:

The Flask web application was successfully containerized using Docker by creating a Dockerfile and building the `myapp:1.0` image. The container was run with port mapping, and the application was successfully accessed using `curl` on port 5000. Container logs were verified, and the container was stopped and removed successfully.

Expected Output

The Docker image should build successfully, the container should run in the background, the application should be reachable on the mapped host port, and docker ps should list the running container. If Docker Hub is used, the tagged image should be visible in the repository.

Conclusion

The application was containerized by writing a Dockerfile, building a Docker image and running it as an isolated portable container, demonstrating image creation, container execution, port mapping and basic container lifecycle management.

Viva / Review Questions

1. Differentiate a container from a virtual machine.
2. What is the difference between a Docker image and a Docker container?
3. Explain the purpose of EXPOSE in a Dockerfile and the -p flag in docker run.
4. Why are Docker image layers cached?

Marks: ____/ 10 Date: _____ Signature of Faculty: _____